Analysis of Input Sensitivity in the Quickhull Algorithm: Recursion Depth and Runtime Performance

Benjamin Stone
Weber State University
Ogden, Utah, USA
benjaminstone2@mail.weber.edu

Abstract

The Quickhull algorithm is a widely used method for computing the convex hull of a set of points, valued for its average-case time complexity of $O(n \log n)$. However, its performance is known to degrade based on input geometry. This study investigates the impact of spatial distribution on both the execution time and the recursion depth of the algorithm. By instrumenting a C++ implementation and testing it against Uniformly Distributed and Circularly Distributed datasets, this research identifies a critical performance divergence. The results demonstrate that while circular inputs degrade the runtime to $O(n^2)$ due to the inability to discard points, they surprisingly maintain an optimal $O(\log n)$ recursion depth due to geometric symmetry.

1 Motivation and Background

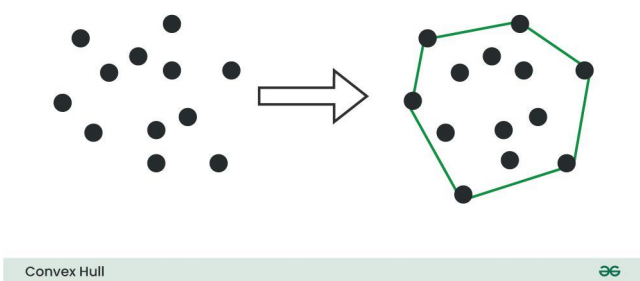


Figure 1: Visual representation of a Convex Hull [2].

The Convex Hull of a set of points S is defined as the smallest convex set that contains the points in S. It is a fundamental construction for mathematics and computational geometry, with applications in mesh generation, file searching, cluster analysis, collision detection, crystallography, metallurgy, urban planning, cartography, image processing, numerical integration, statistics, sphere packing, and point location. [1].

The Quickhull algorithm is a standard method for this task. It operates on a divide-and-conquer principle similar to Quicksort. The algorithm works by selecting a "pivot" point furthest from a baseline, partitioning the set, and recursively solving the sub-problems. While efficient on average, the specific input configurations can satisfy balance conditions to guarantee $O(n \log n)$ performance, while others may degrade performance into $O(n^2)$ behavior[1].

1.1 Research Question

How does the spatial distribution of input coordinates (specifically Uniform vs. Circular) impact the recursion depth and runtime performance of the Quickhull algorithm?

Understanding this relationship is vital for engineering systems where input data might inherently follow a curve (e.g., LiDAR scans of cylindrical objects), as it dictates whether the bottleneck will be CPU time or stack memory.

2 Implementation

To investigate this question, I implemented the Quickhull algorithm in C++. The implementation follows the standard recursive definition but includes specific instrumentation to measure internal behavior.

2.1 Core Logic

The algorithm takes a set of points and two endpoints P_1, P_2. It identifies a point that maximizes the perpendicular distance to the line segment $\overline{P_1P_2}$. The max point is added to the hull, and the problem is split into two recursive calls for the edges $\overline{P_1P_{max}}$ and $\overline{P_{max}P_2}$.

2.2 Instrumentation

To measure the recursion depth without external profiling tools, I adjusted the recursive function to accept a depth parameter:

```
void quickHullRecursive(..., int current_depth) {
    if (current_depth > max_depth) {
        max_depth = current_depth;
    }
    // ... logic ...
    quickHullRecursive(..., current_depth + 1);
}
```

2.3 Test Data Generation

Two datasets were generated for stress testing ($n = 5000$):

- **Uniform Distribution:** Points generated randomly within a 1000×1000 coordinate space. This represents the "Average Case."
- **Circular Distribution:** Points placed equidistantly along the circumference of a circle. This represents a theoretical "Worst Case" where every point belongs to the hull.

3 Algorithm Analysis

The time complexity of Quickhull is described by the recurrence:

$$T(n) = T(n_L) + T(n_R) + f(n)$$

where n_L and n_R are the sizes of the sub-problems, and $f(n)$ is the cost of the partitioning step.

3.1 Partitioning Cost

In this implementation, the partitioning step iterates over the entire original point set to find the farthest point. Therefore, $f(n) = O(N)$ for every recursive step, regardless of the sub-problem size.

3.2 Uniform Analysis

For random points, the "triangle" formed by the pivot and endpoints captures a significant portion of the remaining points. These internal points are discarded. This rapid pruning results in a sparse recursion tree with very few total nodes, leading to $O(N \log N)$ or even $O(N)$ performance in practice.

3.3 Circular Analysis

In a circular arrangement, all points are on the hull.

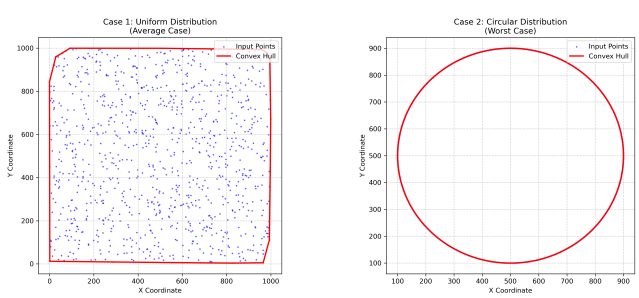
- **No Pruning:** No points are ever strictly "inside" a triangle formed by hull vertices. The algorithm must visit every point.
- **Balanced Tree:** Geometrically, the point furthest from a chord on a circle is the midpoint of the arc. This splits the remaining points evenly $(n/2, n/2)$.

This creates a crucial distinction: The recursion tree is balanced (depth $O(\log N)$), but the total work is quadratic. Since there are $O(N)$ hull points, there are $O(N)$ recursive calls. If each call performs an $O(N)$ scan to find the pivot, the total complexity is:

$$\text{Total Work} \approx N \times O(N) = O(N^2)$$

4 Results

The experiments were conducted using the compiled C++ implementation. The input size was fixed at $N = 5,000$.



(a) Uniform Distribution **(b) Circular Distribution**

Figure 2: Visual comparison of the two experimental datasets. (Images generated using Python with the Scipy ConvexHull algorithm and matplotlib)

Table 1: Performance Comparison ($N = 5000$)

Metric	Uniform (Avg)	Circular (Worst)
Execution Time	0.005 s	1.008 s
Max Recursion Depth	7	13
Hull Points Found	23	5,000

4.1 Discussion

The data (Table 1) reveals two key findings:

1. Runtime Degradation: The circular dataset took approximately 1.0 seconds to process, compared to 0.005 seconds for the uniform dataset. This 200x slowdown confirms the $O(N^2)$ theoretical prediction. Because no points could be discarded, the algorithm was forced to identify all 5,000 points as hull vertices.

2. Optimal Recursion Depth: Contrary to expectations of worst-case behavior, the recursion depth for the circular input was extremely low (13). Given that $\log_2(5000) \approx 12.3$, a depth of 13 implies a perfectly balanced recursion tree. This validates the analysis that the algorithm splits the circle symmetrically at each step.

The contrast between the quadratic runtime and the logarithmic depth indicates that the performance bottleneck is not the shape of the recursion tree, but the redundant scanning of points at each node.

5 Conclusion

This study confirms that Quickhull is highly sensitive to input distribution, specifically regarding computational work.

While the circular distribution represents a worst-case scenario for execution time ($O(N^2)$), it actually represents a best-case scenario for recursion depth ($O(\log N)$) due to geometric symmetry. This distinction is consistent with the empirical evidence provided by Barber et al., who noted that Quickhull processes fewer interior points than randomized algorithms but is sensitive to the distribution of extreme points [1].

This finding suggests that for applications involving circular or convex data, the risk is high latency, not stack overflow. Future work could improve the algorithm by passing filtered subsets of points to recursive calls, ensuring the cost per node scales with the sub-problem size.

References

[1] C Bradford Barber, David P Dobkin, and Hannu Huhdanpaa. 1996. The Quickhull algorithm for convex hulls.

[2] GeeksforGeeks. 2024. Quickhull Algorithm for Convex Hull. https://www.geeksforgeeks.org/dsa/quickhull-algorithm-convex-hull/. Accessed: 2025-12-05.